

CA Agent — Final Year Project

Step-by-Step Extension & Deployment Roadmap

From existing prototype to deployed, evaluable product

Hardware: Intel i5-12th gen · 16 GB RAM · 4 GB VRAM

Project Overview

The CA Agent is a domain-specific AI assistant for Chartered Accountants and their clients. The existing system combines:

- A **React + TypeScript** frontend with Tailwind CSS and shadcn/ui.
- A **FastAPI** backend (`api.py`) with Gemini Vision document extraction (`file_handler.py`).
- A **RAG pipeline** using three precomputed `.joblib` embedding corpora (Foundation, Intermediate, Final) with `bge-m3` via Ollama and a CrossEncoder reranker.
- Optional HyDE query expansion and a metrics module (currently gated by environment flags).

This roadmap extends the project across three phases — twelve discrete steps — and concludes with full cloud deployment.

What fits in 4 GB VRAM: `all-MiniLM-L6-v2` (80 MB), CrossEncoder MiniLM (90 MB), fine-tuning `all-MiniLM-L6-v2` with batch size 16.

What does not: Full `bge-m3` (570 MB, use CPU mode via Ollama instead), any 7B+ local LLM (offload to Gemini Flash API — 1 M free tokens/month).

Rule of thumb: All ML training and inference in this roadmap runs on CPU or uses Gemini Flash API. GPU is only used optionally for faster fine-tuning.

Phase 1 — Foundation Upgrades (Weeks 1–3)

These changes fix known limitations in the existing codebase and add high-visibility features with minimal risk.

2.1. Step 1: Long-term Memory via Rolling Summary

Problem: `api.py` currently builds chat history from only the last 3 completed turns. In a tax planning session that spans 20+ messages, the agent loses context of uploaded documents and earlier clarifications.

Concrete example

Rahul uploads his HDFC bank statement in message 1. Ten messages later he asks: *“Based on what I uploaded, am I eligible for the 80C deduction?”* With the 3-turn window, the agent has no memory of the statement. With rolling summary, the key facts — income **Rs. 9.2L**, TDS **Rs. 48K**, PF deducted — are always in context.

Why it matters: This is a direct fix to a limitation you explicitly documented. Tax planning is a long-form conversation; the app must remember what the user said 10 messages ago.

Implementation:

```
# api.py --- replace last_3_turns() with rolling_context()
import google.generativeai as genai

def summarise_old_turns(history: list[dict]) -> str:
    if len(history) <= 6:
        return ""
    old = history[:-6]
    prompt = f"Summarise these CA conversation turns in 5 bullet points:\n{old}"
    model = genai.GenerativeModel("gemini-1.5-flash")
    return model.generate_content(prompt).text

# In /ask handler:
summary = summarise_old_turns(request.history)
recent = request.history[-6:] # keep last 3 turns verbatim
```

Frontend change: In ChatContext.tsx, store conversation.summary and pass it to /ask alongside history.

2.2. Step 2: Multi-Document Cross-Analysis

Problem: Uploaded documents are currently analysed in isolation. Tax liability, advance tax, and GST returns all require aggregated data across months.

Concrete example

A user uploads ICICI statements for April, May, and June. The app now shows: total income **Rs. 2.7L** over the quarter, average monthly spending **Rs. 38K**, and a flagged anomaly — an unusual **Rs. 80K** outflow in May that does not appear in other months.

Why it matters: CAs never look at a single month in isolation. This transforms the app from a “one document reader” into a genuine financial analysis tool.

Implementation:

```
# ca_agent.py --- new function
def merge_doc_data(docs: list[dict]) -> dict:
    total_credits = sum(d.get("total_credits", 0) for d in docs)
    total_debits = sum(d.get("total_debits", 0) for d in docs)
    monthly = [
        {"month": d["statement_period"], "income": d["total_credits"]}
        for d in docs
    ]
    return {"total_credits": total_credits,
            "total_debits": total_debits,
            "monthly_breakdown": monthly}
```

Frontend: Add a recharts BarChart component in the new dashboard screen showing monthly

income vs. expenses.

2.3. Step 3: Anomaly Detection on Transactions

Problem: Structured extraction surfaces numbers, but does not flag what is unusual. Clients and CAs miss anomalies that affect tax liability.

Concrete example

Priya's statement shows 11 regular monthly credits of approximately **Rs. 45K** and one credit of **Rs. 4.2L**. The app flags: *"Unusual credit on 14 Nov — Rs. 4.2L (8.5× your average). This may be one-time income affecting your advance tax Q3 payment."* Without this, the user might never ask about it.

Why it matters: This is a genuine ML contribution beyond RAG. It adds a proactive intelligence layer and introduces a second, independently evaluable ML component for the thesis.

Implementation:

```
# ca_agent.py
from sklearn.ensemble import IsolationForest
import numpy as np

def detect_anomalies(transactions: list[dict]) -> list[dict]:
    amounts = np.array([t["amount"] for t in transactions]).reshape(-1, 1)
    clf = IsolationForest(contamination=0.05, random_state=42)
    labels = clf.fit_predict(amounts) # -1 = anomaly, 1 = normal
    return [t for t, lbl in zip(transactions, labels) if lbl == -1]
```

Runs entirely on CPU. No GPU needed. Output is appended to `summary_message` in the `/upload` response.

2.4. Step 4: RAGAS Evaluation Harness

Problem: `metrics.py` exists but is gated by `ENABLE_METRICS=true` and never used systematically. There is no quantitative measure of retrieval quality.

Concrete example

You create 100 question-answer pairs from ICAI study material: *"What is the due date for GSTR-3B for a monthly filer?"* → *"20th of the following month."* Running the pipeline against all 100 gives: faithfulness 0.82, answer relevance 0.79, context recall 0.71. After fine-tuning embeddings in Step 6, context recall improves to 0.89. This is your key thesis result.

Why it matters: Every examiner will ask "how do you know it works?" A RAGAS benchmark is your quantitative answer. Before-and-after scores for Steps 6 and 8 form the empirical core of your written report.

Implementation:

```
# run_eval.py (new file)
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_recall
from datasets import Dataset

with open("eval_dataset.json") as f:
    data = json.load(f) # 100 {question, ground_truth, contexts, answer}

dataset = Dataset.from_list(data)
```

```
result = evaluate(dataset, metrics=[faithfulness, answer_relevancy,
                                   context_recall])
print(result)
```

Build `eval_dataset.json` from ICAI PDFs using Gemini Flash to generate Q&A pairs automatically.

2.5. Step 5: CA Analytics Dashboard

Problem: A chat interface is invisible to a non-technical examiner. The data extraction pipeline needs a visual output that makes its value immediately obvious.

Concrete example

After uploading 3 months of statements, the user clicks “Dashboard” and sees: a bar chart of monthly income vs. expenses, a donut chart of TDS deducted vs. total tax owed, and a prominent alert — *“Your advance tax Q2 payment (Rs. 12,400) is due in 15 days.”*

Why it matters: This is your best demo moment. Examiners can see the system working without reading a chat log.

Implementation:

- New route `/dashboard` in `App.tsx`.
- New `Dashboard.tsx` component using `recharts` (`BarChart`, `PieChart`).
- Data sourced directly from `ChatContext` state (`uploadedDocs`).
- No new backend endpoint required.

Phase 2 — Core ML Extensions (Weeks 4–8)

These steps constitute the primary academic and technical contributions.

3.1. Step 6: Fine-Tuned Embeddings on ICAI Corpus

Problem: The current `bge-m3` embeddings are general-purpose. Legal-financial terminology in ICAI material (e.g. “presumptive taxation”, “Section 44AD”, “GSTR-3B late fee”) is not well-represented in the embedding space.

Concrete example

Before fine-tuning: Query *“penalty for late GST filing”* retrieves a generic chunk about penalties.

After fine-tuning: Retrieves the exact GSTR-3B late fee provision (**Rs. 50/day** for non-nil return, **Rs. 20/day** for nil return) because the model has learned that “penalty + GST + late” cluster together in ICAI text. RAGAS context recall improves from 0.71 to 0.89.

Why it matters: This is the single highest-value academic contribution. You are adapting a pretrained model to a specialised legal-financial domain. The before/after RAGAS numbers are your primary thesis result.

Implementation:

```
# fine_tune_embeddings.py (new file)
from sentence_transformers import SentenceTransformer, InputExample, losses
from torch.utils.data import DataLoader

# Build training pairs from ICAI PDFs using Gemini Flash
# Each pair: (question, relevant_chunk)
train_examples = [
    InputExample(texts=["What is Section 44AD?",
                        "Section 44AD allows presumptive taxation at 8% of
                          turnover..."]),
    # ... 500+ pairs
]

model = SentenceTransformer("all-MiniLM-L6-v2") # 80 MB --- fits in 4 GB VRAM
loader = DataLoader(train_examples, shuffle=True, batch_size=16)
loss = losses.MultipleNegativesRankingLoss(model)

model.fit(train_objectives=[(loader, loss)], epochs=3,
          warmup_steps=100, output_path="models/icai-embeddings")
```

After training, update `search.py` to load `models/icai-embeddings` instead of `bge-m3`. Regenerate and save the three `.joblib` files with the new embeddings. Re-run RAGAS to measure improvement.

3.2. Step 7: Agentic ITR Filing Assistant

Problem: The app currently answers questions reactively. ITR filing is a multi-step structured task that benefits from agent-style orchestration.

Concrete example

User uploads salary slips and bank statement.

Step 1 (classify): *"Income Rs. 7.2L, no business income → ITR-1."*

Step 2 (gap-fill): *"I need your home loan interest certificate for 80C. Do you have it?"*

Step 3 (pre-fill): All extractable fields populated from `doc_data`.

Step 4 (export): Pre-filled ITR-1 summary PDF with a checklist of what still needs manual entry.

Why it matters: Every examiner and every real user understands the pain of ITR filing. An agent that does structured work — not just answers questions — is the difference between a prototype and a product.

Implementation:

```
# ca_agent.py --- new class
class ITRAgent:
    STEPS = ["classify", "gap_fill", "pre_fill", "export"]

    def __init__(self, doc_data: dict):
        self.doc_data = doc_data
        self.state = {"step": 0, "itr_type": None, "missing_fields": []}

    def run_step(self, user_input: str = None) -> dict:
        step = self.STEPS[self.state["step"]]
        if step == "classify":
            self.state["itr_type"] = self._classify_itr()
            self.state["step"] += 1
            return {"message": f"Based on your income, you need {self.state['itr_type']}.",
                    "itr_type": self.state["itr_type"]}
        # ... additional steps
```

New endpoint `POST /itr` in `api.py`. New `ITRWizard.tsx` component in the frontend.

3.3. Step 8: GraphRAG on Tax Law

Problem: Pure vector retrieval treats the corpus as independent chunks. Tax law is a dense web of cross-references: Section 80C refers to Section 10(14), which constrains Section 44ADA, and so on.

Concrete example

Query: *“Can I claim HRA and home loan deduction simultaneously?”*

Vector only: Retrieves Section 10(13A) (HRA exemption).

GraphRAG: Traverses the reference edge from 10(13A) to Section 24 (home loan interest) and Section 80C (principal repayment). The answer now covers all three sections and how they interact — the correct, complete answer a CA would give.

Why it matters: This is academically novel. Most RAG projects treat the corpus as independent documents. Graph traversal enables multi-hop reasoning. Runs entirely on CPU using `networkx`.

Implementation:

```
# build_tax_graph.py (new file)
import networkx as nx, re, joblib

G = nx.DiGraph()
# Parse ICAI PDFs and extract section cross-references
# e.g. "as per Section 80C" found in Section 10(13A) text
for section, text in icai_sections.items():
    G.add_node(section)
    refs = re.findall(r"Section\s+[\d\w\(\)]+", text)
    for ref in refs:
        G.add_edge(section, ref)
joblib.dump(G, "tax_graph.joblib")

# search.py --- blend graph + vector retrieval
def graph_retrieve(seed_section: str, hops: int = 2) -> list[str]:
    G = joblib.load("tax_graph.joblib")
    neighbours = nx.single_source_shortest_path_length(G, seed_section, cutoff=hops)
    return list(neighbours.keys())
```

3.4. Step 9: Hallucination Detection and Source Badges

Problem: Incorrect tax advice has real legal consequences. There is currently no mechanism to distinguish retrieval-grounded answers from model-generated inference.

Concrete example

Agent answers: *“Section 44AD allows presumptive taxation at 8% of turnover.”*

NLI model checks this claim against the top retrieved chunk — finds direct textual entailment
→ shows badge **ICAI-verified (Foundation Study Material, Ch. 7)**.

If instead the agent outputs a wrong threshold (e.g. 6% instead of 8%), NLI finds contradiction
→ shows badge **Inferred — verify with your CA**.

Why it matters: This is the trust feature that no other student project will have. For a CA tool specifically, it is critical: it shows you thought about reliability, not just capability.

Implementation:

```
# hallucination_guard.py (new file)
from sentence_transformers import CrossEncoder

nli = CrossEncoder("cross-encoder/nli-MiniLM2-L6-H768") # ~90 MB, CPU-friendly

def check_grounding(answer: str, chunks: list[str]) -> dict:
    pairs = [(chunk, answer) for chunk in chunks]
    scores = nli.predict(pairs) # [entailment, neutral, contradiction]
    best = scores[:, 0].argmax() # entailment column
    if scores[best, 0] > 0.7:
        return {"verified": True, "source": chunks[best][:80]}
    return {"verified": False, "source": None}
```

The returned badge is attached to the /ask response and rendered in `ChatMessage.tsx`.

3.5. Step 10: Compliance Calendar and Deadline Alerts

Problem: The app is purely reactive — users must ask questions to get information. GST and income tax deadlines are predictable from a user's financial profile and should be surfaced proactively.

Concrete example

System detects from uploaded docs: registered GST dealer, turnover **Rs. 1.2Cr**, monthly filer. Automatically generates personalised calendar: GSTR-1 due 11th of each month, GSTR-3B due 20th, advance tax Q3 due 15 Dec, ITR due 31 July. User receives a browser push notification 3 days before each deadline.

Why it matters: Proactive features are rare in student projects. They demonstrate that structured extraction is correct — the calendar only works if `doc_data` is accurate.

Implementation:

```
# compliance_calendar.py (new file)
from datetime import date, timedelta

GST_RULES = {
    "monthly_gstr1": {"day": 11},
    "monthly_gstr3b": {"day": 20},
}

def generate_calendar(doc_data: dict) -> list[dict]:
    deadlines = []
    if doc_data.get("gst_registered"):
        for rule_name, rule in GST_RULES.items():
            for month in range(1, 13):
                deadlines.append({
                    "name": rule_name,
                    "date": date(date.today().year, month, rule["day"]).isoformat()
                })
    return deadlines
```

Frontend: service worker for push notifications, new `CalendarView.tsx` component.

3.6. Step 11: Voice Interface

Problem: The app requires typed input. India's CA client base skews older and less comfortable with keyboards. Tax queries are often long and complex to type.

Concrete example

A 55-year-old business owner taps the mic button and says: *“Mera GST kitna banega is mahine?”* The app transcribes via the Web Speech API, routes to the CA agent, and reads the answer aloud using `SpeechSynthesis`. Zero backend changes required.

Why it matters: Voice accessibility is a 50-line frontend change with outsized demo impact. Examiners will remember it. It demonstrates consideration for the real target user, not just the tech-savvy demo audience.

Implementation:

```
// ChatInput.tsx --- add voice hook
const startListening = () => {
  const recognition = new window.webkitSpeechRecognition();
  recognition.lang = "en-IN"; // or "hi-IN" for Hindi
  recognition.onresult = (e) => {
    setInput(e.results[0][0].transcript);
  };
  recognition.start();
};

// ChatMessage.tsx --- add read-aloud button
const speak = (text) => {
  const utter = new SpeechSynthesisUtterance(text);
  utter.lang = "en-IN";
  window.speechSynthesis.speak(utter);
};
```

Phase 3 — Deployment (Weeks 9–12)**4.1. Step 12: Remove Ollama Dependency**

Problem: Ollama is a local development convenience. In cloud environments (Render, Railway), `http://localhost:11434` does not exist. Every embedding call will fail.

Concrete example

On Render.com, the backend starts and immediately calls `http://localhost:11434/api/embeddings`. That server does not exist in the cloud. Every user query returns a 500 error. Replacing with `model.encode(query)` means embedding runs in-process — no sidecar, no configuration, works everywhere.

Implementation:

```
# search.py --- replace the Ollama HTTP call
# BEFORE:
# r = requests.post("http://localhost:11434/api/embeddings",
#                  json={"model": "bge-m3", "prompt": query})
# embedding = r.json()["embedding"]

# AFTER:
from sentence_transformers import SentenceTransformer
_model = SentenceTransformer("models/icai-embeddings") # fine-tuned in Step 6

def get_embedding(text: str) -> list[float]:
    return _model.encode(text, normalize_embeddings=True).tolist()
```


Remove `OLLAMA_URL` from `dev_config.py` and `.env`.

4.2. Step 13: Dockerise the Backend

Problem: Without containerisation, reproducibility depends on a specific Python version, installed packages, and the presence of `.joblib` files in exactly the right paths.

Concrete example

Without Docker: a reviewer tries to run the project, hits “*Ollama not found*”, then “*joblib version mismatch*”, then “*sentence_transformers not installed*.” With Docker: `docker compose up` and the full system is live in 2 minutes — backend, all dependencies, and `.joblib` files all packaged together.

Implementation:

```
# Dockerfile
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY *.joblib .
COPY models/ models/
COPY *.py .
EXPOSE 8000
CMD ["uvicorn", "api:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
# docker-compose.yml
services:
  backend:
    build: .
    ports: ["8000:8000"]
    env_file: .env
  frontend:
    build: ./frontend
    ports: ["5173:5173"]
    depends_on: [backend]
```

4.3. Step 14: Cloud Deployment

Two options depending on need:

Option A — Free cloud (recommended for submission)

1. Backend on Render.com (free tier):

- Connect GitHub repo. Render detects `Dockerfile` automatically.
- Set environment variables in Render dashboard (`GEMINI_API_KEY`, etc.).
- `.joblib` files either committed to repo or pulled from Hugging Face Hub at startup.
- Note: free tier spins down after 15 minutes of inactivity. For always-on, use Railway.app (\$5/month credit is usually sufficient).

2. Frontend on Vercel (free):

- `vercel -prod` from the `frontend/` directory.

- Set `VITE_API_URL` to your Render backend URL.
- Build command: `npm run build`. Output directory: `dist/`.

Option B — Local with tunnel (best for demo day)

1. Run backend: `uvicorn api:app -host 127.0.0.1 -port 8000`
2. Run frontend: `npm run dev`
3. Expose via Cloudflare Tunnel (free): `cloudflared tunnel -url http://localhost:8000`
4. Share the generated `*.trycloudflare.com` URL with the panel.

- `.env` is in `.gitignore` — never committed.
- `.joblib` files: if over 100 MB, use Git LFS or Hugging Face Hub (`huggingface_hub.upload_file()`).
- CORS in `api.py`: restrict `allow_origins` to your Vercel URL in production.
- `ENABLE_HYDE` and `ENABLE_METRICS` set to `false` in production to reduce latency.

Summary: All Steps at a Glance

Step	Phase	Feature	Key file(s)
1	1	Rolling conversation memory	<code>api.py</code> , <code>ChatContext.tsx</code>
2	1	Multi-document cross-analysis	<code>ca_agent.py</code> , <code>Dashboard.tsx</code>
3	1	Anomaly detection (IsolationForest)	<code>ca_agent.py</code>
4	1	RAGAS evaluation harness	<code>run_eval.py</code> , <code>eval_dataset.json</code>
5	1	CA analytics dashboard	<code>Dashboard.tsx</code> , <code>App.tsx</code>
6	2	Fine-tuned ICAI embeddings	<code>fine_tune_embeddings.py</code> , <code>search.py</code>
7	2	Agentic ITR filing assistant	<code>ca_agent.py</code> , <code>ITRWizard.tsx</code>
8	2	GraphRAG on IT Act sections	<code>build_tax_graph.py</code> , <code>search.py</code>
9	2	Hallucination detection + badge	<code>hallucination_guard.py</code> , <code>api.py</code>
10	2	Compliance calendar + alerts	<code>compliance_calendar.py</code> , <code>CalendarView.tsx</code>
11	2	Voice interface (Web Speech API)	<code>ChatInput.tsx</code> , <code>ChatMessage.tsx</code>
12	3	Remove Ollama, in-process embed	<code>search.py</code>
13	3	Dockerise backend	<code>Dockerfile</code> , <code>docker-compose.yml</code>
14	3	Deploy: Render + Vercel / tunnel	<code>.env</code> , Render dashboard

Suggested Thesis Chapter Mapping

1. **Introduction** — Problem statement, CA domain context, app overview.
2. **Literature Review** — RAG systems, domain-specific NLP, hallucination in LLMs, RAGAS evaluation.

3. **System Architecture** — Existing system (Phase 0), full extended architecture diagram.
4. **Chapter: Retrieval Pipeline** — Steps 6 (fine-tuning) and 8 (GraphRAG). Present RAGAS before/after numbers as Table 4.1.
5. **Chapter: Agent Design** — Steps 7 (ITR agent), 9 (hallucination guard), 10 (compliance calendar). Step-state machine diagram.
6. **Chapter: User Experience** — Steps 2 (multi-doc), 3 (anomaly), 5 (dashboard), 11 (voice). Usability evaluation.
7. **Chapter: Deployment** — Steps 12–14. Architecture diagram, latency measurements, cost analysis.
8. **Evaluation** — RAGAS results, anomaly detection precision/recall, user study (3–5 CA professionals).
9. **Conclusion** — Limitations, future work (multi-language support, mobile app, backend session store).

12-Week Timeline

Week	Primary task	Deliverable
1	Steps 1–2: memory, multi-doc merge	Working /ask with rolling context
2	Step 3: anomaly detection	Flagged transactions in upload response
3	Steps 4–5: RAGAS harness, dashboard	Baseline RAGAS scores, dashboard screen
4	Step 6: collect ICAI Q&A pairs	500+ training pairs JSON
5	Step 6: fine-tune + re-embed	New .joblib files, improved RAGAS
6	Step 7: ITR agent skeleton	/itr endpoint, classification step working
7	Step 7: ITR agent complete + PDF export	Full 4-step wizard
8	Steps 8–9: GraphRAG + NLI guard	Hybrid retrieval, source badges in UI
9	Steps 10–11: calendar + voice	Push notifications, mic button working
10	Step 12: remove Ollama	In-process embedding, all tests pass
11	Step 13: Docker + local testing	<code>docker compose up</code> runs fully
12	Step 14: deploy + final evaluation	Live Render + Vercel URLs, final RAGAS